Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 23.Б08-мм

Интеграция алгоритма поиска константных условных функциональных зависимостей CFUN в Desbordante

Кожуков Иван Сергеевич

Отчёт по учебной практике
в форме «Производственное задание»

Научный руководитель:
ассистент кафедры ИАС Г.А. Чернышев

Санкт-Петербург
2026

Оглавление

1. Введение **3**

2. Постановка задачи **4**

3. Условные функциональные зависимости **5**

3.1. Поиск CFDs 5

3.2. Поиск CCFDs 6

3.3. Поиск канонического покрытия CCFDs 8

4. Алгоритм CFUN **10**

5. Интеграция CFUN **12**

5.1. Анализ исходной реализации 12

5.2. Ошибки исходной реализации 14

5.3. Новая реализация 15

6. Эксперименты **17**

6.1. Описание экспериментов 17

6.2. Результаты экспериментов 18

7. Заключение **20**

Список литературы **21**

1. Введение

В современную эпоху больших данных обеспечение качества информации стало критически важной задачей для организации различных сфер. Одним из ключевых подходов к решению этой проблемы является автоматизированное обнаружение семантических зависимостей в данных, которые позволяют выявлять противоречия, аномалии и закономерности. Среди множества типов зависимостей особое место занимают условные функциональные зависимости (conditional functional dependencies, далее — CFDs), представляющие собой мощный инструмент для анализа данных и их очистки [1].

CFDs расширяют классические функциональные зависимости (functional dependencies, далее — FDs), позволяя комбинировать атрибуты и конкретные значения в правилах, что делает их более выразительными и практичными для реальных задач.

В научном сообществе выделяют два основных типа CFD: константные CFD (constant CFDs, далее — CCFDs) и переменные CFD (variable CFDs, далее — VCFDs). Каждый из этих типов имеет свою область применения и требует различных алгоритмических подходов для эффективного обнаружения. Например, в статье [4] авторы описывают два подхода к поиску CCFDs: первый подход остается лишь теоретическим результатом, а второй — имеет реализацию на языке C++ в виде алгоритма CFUN.

Desbordante [3] — высокопроизводительный наукоёмкий профилировщик данных, включающий в себя множество алгоритмов для нахождения и проверки различных сложных зависимостей, в том числе CFDs. На момент написания данной работы среди имеющихся алгоритмов отсутствует алгоритм для целенаправленного поиска CCFDs, поэтому для расширения функциональности платформы было принято решение интегрировать CFUN в Desbordante[1] на языке C++.

[1] https://github.com/Desbordante

2. Постановка задачи

Целью работы является интеграция алгоритма поиска константных условных функциональных зависимостей CFUN, представленного авторами статьи [4], в платформу Desbordante. Для её достижения были поставлены следующие задачи:

1. Выполнить обзор предметной области и работы алгоритма CFUN;
2. Интегрировать алгоритм CFUN в профайлер Desbordante;
3. Провести тестирование и сравнение полученной реализации и реализации из статьи [4].

3. Условные функциональные зависимости

В этом разделе приводятся основные определения и утверждения, необходимые для описания алгоритма. Все формулировки взяты из статьи [4]. Будем рассматривать схему отношения R и ее конкретные экземпляры r.

3.1. Поиск CFDs

Определение 1. *CFD φ — это пара $(X \to Y, T_p)$, где $X \to Y$ — стандартная FD и T_p — таблица шаблонов (pattern tableau) из атрибутов $X \cup Y$ такая, что $\forall A \in X \cup Y$ и $\forall t_p \in T_p : t_p[A] = a$, где $a \in dom(A)$, или $t_p[A] =$ "_", где "_" — безымянная переменная (unnamed variable). Кортеж $t_p \in T_p$ называется шаблоном.*

Определение 2. *CFD, шаблоны которой состоят только из констант, называется константной CFD (CCFDs). Если же некоторый шаблон содержит безымянную переменную, то такая CFD называется переменной (VCFD).*

Определение 3. *Кортеж $t \in r$ соответствует шаблону $t_p \in T_p$ на множестве атрибутов $S \subset R$, обозначается $t[S] \asymp t_p[S]$, если $\forall B \in S$: $t_p =$ "_" или $t_p[B] = t[B]$.*

Определение 4. *r соответствует CFD $\varphi = (X \to Y, T_p)$, обозначаем $r \vDash \varphi$, если $\forall t_i, t_j \in r$ и $\forall t_p \in T_p : t_i[X] = t_j[X] \asymp t_p[X] \Longrightarrow t_i[Y] = t_j[Y] \asymp t_p[Y]$.*

Определение 5. *r нарушает CFD $\varphi = (X \to Y, T_p)$, обозначаем $r \nvDash \varphi$, если выполнено одно из условий:*

1. *$\exists t \in r, \exists t_p \in T_p$ такие, что $t[X] \asymp t_p[X]$ и $t[Y] t_p[Y]$;*
2. *$\exists t_1, t_2 \in r, \exists t_p \in T_p$ такие, что $t_1[X] = t_2[X] \asymp t_p[X]$ и $t_1[Y] t_2[Y]$;*

Определение 6. *Будем говорить, что r соответствует множеству CFDs Σ, обозначается $r \models \Sigma$, если $\forall\ \varphi \in \Sigma\ r \models \varphi$.*

Определение 7. *Пусть Σ_1 и Σ_2 — два множества CFD над одной схемой R. Будем говорить, что Σ_1 эквивалентно Σ_2, обозначается $\Sigma_1 \equiv \Sigma_2$, если $\forall\ r$ над R выполнено: $r \vDash \Sigma_1 \Leftrightarrow r \vDash \Sigma_2$.*

Общая задача по поиску CFDs формулируется так: для данного отношения r найти покрытие множества CFDs Σ_r, соответствующих r, то есть такое множество Σ', что $\Sigma' \subseteq \Sigma_r$ и $\Sigma' \equiv \Sigma_r$.

Определение 8 (Нормальная форма)**.** *$\phi = (X \to Y,\ T_p)$ CFD в нормальной форме, если $|Y| = 1$ и $|T_p| = 1$.*

Утверждение 1. *Для любого множества CFDs Σ существует множество CFDs в нормальной форме Σ_{nf} такое, что $\Sigma \equiv \Sigma_{nf}$.*

Утверждение 2. *Для любого множества CFDs Σ существует множество CCFDs Σ_c и множество VCFDs Σ_v такие, что $\Sigma \equiv \Sigma_c \cup \Sigma_v$.*

При помощи приведенных определений формулируется следующее утверждение:

Утверждение 3. *Для любого отношения r существует множество CCFDs Σ_c такое, что $\Sigma_r \equiv \Sigma_c$, где Σ_r — множество CFDs, удовлетворяющих r.*

Тогда, основываясь на утверждении 3 задачу поиска CFDs можно свести к задаче поиска CCFDs следующим образом: для данного отношения r найти покрытие множества CCFDs Σ_c, удовлетворяющих r. Далее будем рассматривать только CCFDs в нормальной форме.

3.2. Поиск CCFDs

Для описания нового подхода к поиску CCFDs в статье [4] вводятся следующие определения:

Определение 9. *Пространство поиска CCFDs —*

$$ASP(R, r) = \{(A, a) \mid A \in R, a \in ADOM(A, r)\},$$

где $ADOM(A, r)$ — фактический домен атрибута A в отношении r.

Таблица 1: Отношение r над схемой $R = \{A, B, C, D\}$

r	A	B	C	D
t_1 :	0	1	0	2
t_2 :	0	1	3	2
t_3 :	0	0	0	1
t_4 :	2	2	0	1
t_5 :	2	1	0	1

Рассмотрим пример $ASP(R, r)$ для отношения из таблицы 1. Будем обозначать пару (A, a) как Aa. Тогда $ASP(ABCD, r) = \{A0, A2, B0, B1, B2, C0, C3, D1, D2\}$.

В дальнейшем будем использовать следующее обозначения: для **пары** $\overline{A} = (A, a) \in ASP(R, r)$ обозначим $\overline{A}.attr$ и $\overline{A}.val$ значения A и a соответственно. Если $\overline{X} \subseteq ASP(R, r)$ — **множество атрибутов**, то $\overline{X}.attr = \bigcup_{\overline{A} \in \overline{X}} \overline{A}.attr$.

Используя эти обозначения можно переписать определение CCFD следующим образом: CCFD $\varphi = (A_1 \dots A_n \to A, t_p[A_1 \dots A_n A])$ синтаксически эквивалентна записи $\overline{X} \to \overline{A}$, где $\overline{X} = \{(A_1, t_p[A_1]), \dots, (A_n, t_p[A_n])\} \subseteq ASP(R, r)$ и $\overline{A} = (A, t_p[A]) \in ASP(R, r)$.

Определение 10 (Нетривиальная CCFD)**.** *CCFD $\overline{X} \to \overline{A}$ называется нетривиальной, если $\overline{A}.attr \notin \overline{X}.attr$.*

Определение 11 (Лево-приведенная CCFD)**.** *CCFD $\overline{X} \to \overline{A}$ называется лево-приведенной над r, если для любого $\overline{Y}.attr \subset \overline{X}.attr$ $r \not\models \overline{Y} \to \overline{A}$.*

Определение 12 (Минимальная CCFD)**.** *ϕ — минимальная CCFD над r, если она нетривиальная, лево-приведенная и $r \models \phi$.*

Определение 13 (Каноническое покрытие)**.** *Σ_{cc} — каноническое покрытие для Σ_r, если Σ_{cc} — множество минимальных CCFD такое, что $\Sigma_r \equiv \Sigma_{cc}$.*

Таким образом, задачу поиска CCFDs можно переформулировать следующим образом: для данного отношения r найти каноническое покрытие множества CCFDs, удовлетворяющих r.

3.3. Поиск канонического покрытия CCFDs

Во избежание поиска CFDs, которые не соответствуют ни одному кортежу r, авторы статьи вводят определение частой CFD.

Рассмотрим оператор выборки (selection) из реляционной алгебры. $\sigma_C(r)$ — оператор, возвращающий все строки в отношении r, удовлетворяющие условию C. Обозначим за $C_{\overline{X}}$ следующее условие: кортеж $t \in r$ подходит под условие $C_{\overline{X}}$, если $\forall (A, a) \in \overline{X}\ t[A] = a$.

Определение 14 (Частота). *Пусть $\varphi = \overline{X} \to \overline{Y}$ — CCFD над r. Частота φ в r:*

$$freq(\varphi, r) = |\sigma_{C_{\overline{X} \bigcup \overline{Y}}}(r)|$$

Частота множества атрибутов $\overline{X} \subseteq ASP(R, r)$:

$$freq(\overline{X}, r) = |\sigma_{C_{\overline{X}}}(r)|$$

Пример вычисления частоты. Рассмотрим $\varphi = A2 \to C0$ — CCFD над отношением из таблицы 1. Тогда оператор выборки $\sigma_{C_{\{A2,C0\}}}(r)$ вернет t_4 и t_5, так как только в этих строках атрибут A принимает значение 2, а атрибут C — значение 0. То есть $freq(\varphi, r) = 2$.

Для множества атрибутов $\overline{X} = \{A2, C0, D1\}$ оператор $\sigma_{C_{\{A2,C0,D1\}}}(r)$ вернет t_4 и t_5, так как только в этих строках атрибут A принимает значение 2, атрибут C — значение 0, а атрибут D — значение 1. То есть $freq(\overline{X}, r) = 2$.

Определение 15 (Частая CFD). *Пусть $\varepsilon \geq 0$ — порог частоты. Будем говорить, что CCFD ϕ над r — ε-частая, если $freq(\phi, r) \geq \varepsilon$.*

Определение 16 (Неизбыточное множество). *Пусть $\overline{X} \subseteq ASP(R, r)$ — множество атрибутов. $\overline{X}$ — неизбыточное множество для r, если $\nexists \overline{X'} \subseteq \overline{X}$ такого, что $freq(\overline{X'}, r) = freq(\overline{X}, r)$.*

Все неизбыточные множества будем обозначать $\mathcal{NRS}_r$. Остальные множества атрибутов будем называть избыточными.

Утверждение 4. *Пусть $\overline{X}, \overline{Y} \subseteq ASP(R, r)$ такие, что $\overline{X} \subseteq \overline{Y}$. Тогда $\overline{Y} \in \mathcal{NRS}_r \Rightarrow \overline{X} \in \mathcal{NRS}_r$. Аналогично, $\overline{X} \notin \mathcal{NRS}_r \Rightarrow \overline{Y} \notin \mathcal{NRS}_r$.*

Определение 17 (Замыкание). *Пусть $\overline{X} \subseteq ASP(R, r)$ — множество атрибутов. Тогда его замыкание (closure), обозначаем $\overline{X}^*_{\Sigma_r}$ — это множество атрибутов такое, что*

$$\overline{X}^*_{\Sigma_r} = \overline{X} \cup \{\overline{A} \mid \overline{A} \in ASP(R, r) \wedge \overline{A}.attr \neq \overline{X}.attr \wedge r \vDash \overline{X} \to \overline{A}\}$$

То есть замыкание множества атрибутов $\overline{X}$ состоит из $\overline{X}$ и всех пар из $ASP(R, r)$, не содержащих атрибут X, которые функционально зависят от $\overline{X}$.

Определение 18 (Квази-замыкание). *Пусть $\overline{X} \subseteq ASP(R, r)$ — множество атрибутов. Тогда его квази-замыкание (quasi-closure), обозначаем $\overline{X}^\diamond_{\Sigma_r}$ — это множество атрибутов такое, что*

$$\overline{X}^\diamond_{\Sigma_r} = \overline{X} \cup \bigcup_{\overline{A} \in \overline{X}} (\overline{X} - \overline{A})^*_{\Sigma_r}$$

То есть квази-замыкание множества атрибутов $\overline{X}$ состоит из $\overline{X}$ и замыканий всех его подмножеств, полученных удалением одной пары.

Замечание. По свойству монотонности замыкания верно утверждение:

$$\overline{X} \subseteq \overline{X}^\diamond_{\Sigma_r} \subseteq \overline{X}^*_{\Sigma_r}$$

С помощью вышеописанных определений формулируется следующая теорема, которая является основой рассматриваемого алгоритма.

Теорема 1 (Каноническое покрытие CCFDs). *Каноническое покрытие множества ε-частых CCFDs для r состоит из всех CCFDs, где левая часть — неизбыточное множество атрибутов с частотой не меньше, чем ε, а правая часть — пара из замыкания $\overline{X}$, которая не входит в квази-замыкание $\overline{X}$, то есть:*

$$\Sigma_{cc_\varepsilon}(R, r) = \{\overline{X} \to \overline{A} \mid \overline{X} \in \mathcal{NRS}_r \wedge \ freq(\overline{X}, r) \geq \varepsilon \wedge \overline{A} \in \overline{X}^*_{\Sigma_r} - \overline{X}^\diamond_{\Sigma_r}\}$$

4. Алгоритм CFUN

Формально, алгоритм CFUN ищет неизбыточные множества и соответствующие им характеристики, что с учетом теоремы 1 делает вывод канонического покрытия CCFDs тривиальным. CFUN наследует идею поуровневого обхода решетки от алгоритма поиска функциональных зависимостей FUN [5]. Каждый уровень решетки состоит из четверок вида $\langle \overline{X}, |\overline{X}|, \overline{X}^{\diamond}_{\Sigma_r}, \overline{X}^{*}_{\Sigma_r} \rangle$, определяющих кандидата, его частоту, замыкание и квази-замыкание соответственно.

Алгоритм 1: CFUN [4]

1 $L_0 \leftarrow \langle \overline{\emptyset}, |r|, \overline{\emptyset}, \overline{\emptyset} \rangle$
2 $L_1 \leftarrow \{\langle \overline{A}, |\overline{A}|, \overline{A}, \overline{A} \rangle \mid \overline{A} \in ASP_{CFD}(R, r) \wedge |\overline{A}.att| = 1\}$
3 **for** $k \leftarrow 1$; $L_k \neq \emptyset$; $k \leftarrow k + 1$ **do**
4 ⎸ ComputeClosures(L_{k-1}, L_k)
5 ⎸ ComputeQuasiClosures(L_k, L_{k-1})
6 ⎸ DisplayCFDs(L_{k-1})
7 ⎸ PruneRedundantSets(L_k, L_{k-1})
8 ⎸ $L_{k+1} \leftarrow$ GenerateCandidates(L_k)
9 **end**
10 ComputeClosures(L_{k-1}, L_k)
11 DisplayCFDs(L_{k-1})

Алгоритм начинается с формирования нулевого и первого уровня (строки 1–2). Нулевой уровень содержит единственного кандидата — пустое множество атрибутов, а первый уровень образуется из кандидатов, составленных из одиночных атрибутов отношения. Первоначально в замыкание и квази-замыкание каждого кандидата первого уровня добавляется сам кандидат.

На каждом шаге цикла вычисляется замыкание кандидатов, оставшихся на прошлом уровне, а затем — квази-замыкание кандидатов на текущем уровне. CCFDs, соответствующие r, выводятся по теореме 1 (строка 6). Из текущего уровня удаляются избыточные кандидаты в соответствии с утверждением 4 (строка 7), а затем из него формируется новый уровень через объединение кандидатов. Цикл

останавливается, если новый сформированнй уровень оказывается пустым. Алгоритм завершается выводом CCFDs, обнаруженных на предпоследнем сформированном уровне.

На практике для вычисления частоты используется понятие партиции (partitions [2]). В контексте алгоритма партиция объединяет строки отношения r с одним набором значений в $\overline{X}.val$ в кластеры по всем возможным наборам $\overline{X}$.

Пример. В таблице 2 представлено несколько итераций алгоритма для отношения 1. Звездочкой $*$ помечены избыточные кандидаты. Зависимость вида $A2 \rightarrow C0D1$, где в правой части более одного атрибута, можно разложить на зависимости $A2 \rightarrow C0$ и $A2 \rightarrow D1$.

Таблица 2: Пример итераций алгоритма.

Уровень	X	$\|X\|$	$X^{\diamond}_{\Sigma}$	X^{*}_{Σ}	CCFDs
0	$\emptyset$	5	$\emptyset$	$\emptyset$	
1	A0	3	A0	A0	
	A2	2	A2	A2 C0 D1	A2 → C0D1
	B1	3	B1	B1	
	B0	1	B0	A0 B0 C0 D1	B0 → A0C0D1
	B2	1	B2	A2 B2 C0 D1	B2 → A2C0D1
	C0	4	C0	C0	
	C3	1	C3	A0 B1 C3 D2	C3 → A0B1D2
	D2	2	D2	A0 B1 D2	D2 → A0B1
	D1	3	D1	C0 D1	D1 → C0
2	A0 B1	2	A0 B1	A0 B1 D2	A0B1 → D2
	*A0 B0	1	A0 B0 C0 D1	A0 B0 C0 D1	
	A2 B1	1	A2 B1 C0 D1	A2 B1 C0 D1	
	*A2 B2	1	A2 B2 C0 D1	A2 B2 C0 D1	
	A0 C0	2	A0 C0	A0 C0	
	*A0 C3	1	A0 B1 C3 D2	A0 B1 C3 D2	
	*A2 C0	2	A2 C0 D1	A2 C0 D1	
	*A0 D2	2	A0 B1 D2	A0 B1 D2	
	A0 D1	1	A0 C0 D1	A0 B0 C0 D1	A0D1 → B0
	*A2 D1	2	A2 C0 D1	A2 C0 D1	
	...	...	...	...	
3	*A0 D1 C0	1	A0 D1 C0 B0	A0 D1 C0 B0	
	...	...	...	...	

5. Интеграция CFUN

5.1. Анализ исходной реализации

Как было упомянуто ранее, авторы статьи [4] опубликовали собственную реализацию[2] алгоритма на языке C++. Автором настоящей работы была рассмотрена возможность заимствования исходного кода. Однако, при анализе реализации авторов статьи был установлен ряд недостатков, среди них можно выделить: использование неинформативных имен для переменных и классов, дублирование функционала стандартной библиотеки, а также нарушение принципа инкапсуляции. В качестве примера рассмотрим следующий фрагмент кода:

```
bool LevelCFUN::Init(unsigned int uNbElemAlloc) {
    if (m_pListeAttr) {
        DeleteListe();

        if (uNbElemAlloc <= m_uNbElemAlloc) {
            ::memset(m_pListeAttr, 0,
                     sizeof(*m_pListeAttr) * m_uNbElemAlloc);
            return true;
        }

        ::free(m_pListeAttr);
    }

    m_uNbElemAlloc = uNbElemAlloc;
    m_pListeAttr = (PartitionCFUN*)::malloc(
        sizeof(*m_pListeAttr) * m_uNbElemAlloc);

    if (!m_pListeAttr)
        return false;

    ::memset(m_pListeAttr, 0,
             sizeof(*m_pListeAttr) * m_uNbElemAlloc);

    return true;
}
```

[2] https://pageperso.lis-lab.fr/~noel.novelli/CFDProject/ (дата доступа: 8 февраля 2026г.)

Приведенный фрагмент содержит метод `Init()`, который выделяет память для полей объекта `LevelCFUN`, представляющий уровень решетки.

В современных стандартах C++ настоятельно рекомендуется избегать ручного выделения памяти, и тем более смешение оператора `new` и функций языка C для работы с памятью, как происходит в некоторых частях программы. Также приведенный метод, содержит логику инициализации, которая должна быть перемещена в конструктор объекта `LevelCFUN`.

Аналогичные проблемы присутствуют в классе `PartitionCFUN`, многие поля которого инициализируются с помощью отдельных методов вне конструктора. Также из названия класса `PartitionCFUN` можно сделать вывод, что он представляет собой партицию для кандидата $\overline{X}$. Однако, на самом деле этот класс представляет четверку из кандидата и его характеристик, а партиция содержится в классе отдельным полем.

К существующим недостаткам можно отнести особенность реализации, которая накладывает ограничение на входную таблицу. В исходном коде для хранения множества атрибутов используется класс-обертка над целочисленной переменной типа `unsigned int`, где атрибуты отображаются в соответствующие биты. Кроме того, что такое решение вынуждает создавать собственные реализации стандартных методов класса `std::bitset`, оно так же ограничивает максимально возможное число атрибутов в таблице (то есть на большинстве современных систем алгоритм не сможет обработать таблицу с 32 и более атрибутами).

Дополнительно отметим, что в исходной реализации все обнаруженные CCFDs выводятся непосредственно в консоль через `std::cout`. Однако, согласно философии Desbordante, все алгоритмы должны предоставлять возможность программного доступа к своим результатам через Python-интерфейс. Поэтому для полноценной интеграции алгоритма в платформу необходимо связать код алгоритма на C++ с Python-библиотекой.

5.2. Ошибки исходной реализации

Помимо проблем с качеством кода исходная реализация содержит несколько ошибок в логике, которые могут повлиять на результаты алгоритма.

Первая ошибка заключается в неправильной работе парсера класса `Relation`. Если домен некоторого атрибута состоит из строк и некоторая из них содержит символ, совпадающий с символом-разделителем таблицы, то при построчной обработке таблицы парсер может досрочно закончит свою работу. При этом программа не завершится аварийно, а начнет выполнять алгоритм для частично обработанной таблицы. Это происходит из-за ошибки в реализации метода `ReadTextFile::splitline`, которая считает внутренний символ-разделитель в строке за начало значения следующего столбца.

Для описания второй ошибки рассмотрим следующий фрагмент кода:

```
1  void cfun(...) {
2  ///
3    LevelCFUN &level1 = levels[1];
4    level1.m_uNbAttr = relation.getNbCol();
5    if (!level1.Init(level1.m_uNbAttr)) {
6      cerr << "Allocation error CListePartition.m_pListeAttr" << endl;
7      exit(-5);
8    }
9    for (unsigned int i = 0; i < level1.m_uNbAttr; ++i) {
10     level1.m_pListeAttr[i] = relation.getPartitionsMonoDim(i);
11   }
12 ///
13 }
```

Приведенная часть кода инициализирует кандидатов на первом уровне решетки. Метод `getPartitionsMonoDim()` класса `Relation` извлекает кластеры, которые были построены для каждого одиночного атрибута при обработке входной таблицы. Однако у класса `PartitionCFUN` также имеется поле `m_pListeSubset`, которое для кандидата $\overline{X}$ хранит указатели на кандидаты-подмножества

$\overline{A} \subset \overline{X}$ с нижнего уровня. Для инициализации этого поля используется функция `initSubset()`, которая вызывается только при создании новых кандидатов в `LevelCFUN::GenerateNextLevel()`, но не при создании кандидатов первого уровня. Из-за этого для первого уровня всегда пропускается этап обновления квази-замыкания, что может привести к неправильной работе алгоритма, если на нулевом уровне будет обнаружена зависимость. Все выявленные недостатки и ошибки были исправлены в новой реализации.

5.3. Новая реализация

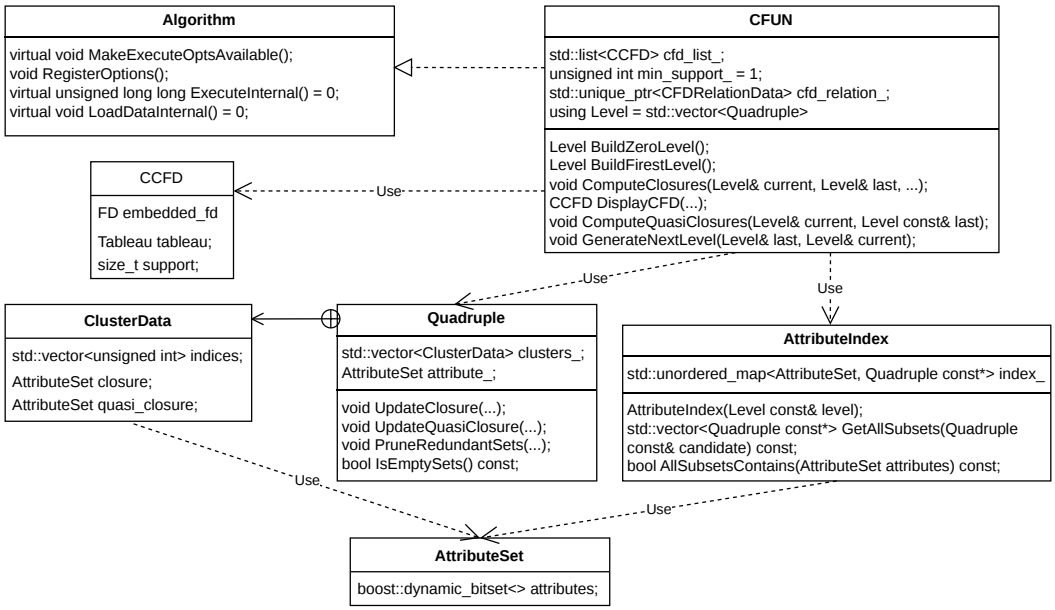

Рис. 1: UML-диаграмма классов алгоритма CFUN.

На рисунке 1 представлена UML-диаграмма классов новой реализации алгоритма CFUN.

Класс `CFUN` наследуется от базового класса `Algorithm`, который предоставляет все необходимые методы для загрузки параметров и входных данных, а также запуска и получения результатов алгоритма. Кроме входной таблицы алгоритм имеет единственный дополнительный параметр `min_sup`, задающий минимальный порог частоты для кандидатов. Вся логика алгоритма выполняется в методе

`ExecuteInternal()`.

Для хранения множества атрибутов используется тип `AttributeSet`, который реализован с помощью `boost::dynamic_bitset<>`, что позволяет обрабатывать таблицы любых размеров.

Класс `Quadruple` представляет четверку для кандитата $\overline{X}$, из которых состоит уровень решетки, реализованный как `std::vector<Quadruple>`. В отличие от формального описания алгоритма `Quadruple` хранит информацию о нескольких кандидатах с одинаковым набором атрибутов X, но разными константами. Такое решение позволяет пакетно обрабатывать кандидатов при вычислении замыканий.

Для хранения набора констант используются партиция, объединяющая строки с одинаковыми значениями на атрибутах $\overline{X}.attr$ в кластеры. Патриция представлена как `std::vector<Cluster>`. Для каждого кластера в классе `Quadruple` хранятся замыкание и квази-замыкание как `AttributeSet`.

Результаты алгоритма сохраняются в `std::list<CCFD>`, где `CCFD` — класс представления результата, который объединяет несколько константных шаблонов над одним набором атрибутов в одну таблицу шаблонов согласно стандартному определению CFDs.

Для связки алгоритма с Python была использована библиотека `pybind11`.

6. Эксперименты

6.1. Описание экспериментов

В таблице 3 описаны два набора данных, на которых проводилось тестирование алгоритма.

Таблица 3: Экспериментальные наборы данных.

Набор	**Столбцы**	**Строки**	**Размер**
mushroom	23	8,124	379 КБ
campus[3]	26	100,000	12.42 МБ

Другие рассмотренные наборы данных не были включены в эксперименты, так как на них обе реализации алгоритма работали за одинаково малое количество времени (менее 5 секунд) даже при небольшом пороге частоты.

Характеристики системы, на которой проводились эксперименты: Intel(R) Core(TM) i7-12700H CPU @ 3.5GHz (400–4700GHz, 14 core 20 threads), 15 GiB RAM, Ubuntu 24.04.2 LTS, Kernel 6.14.0-32-generic, gcc (Ubuntu 13.3.0-6ubuntu2 24.04) 13.3.0.

Для каждого набора данных был проведен следующий эксперимент: было сделано по семь запусков исходной и новой реализации алгоритма, на каждой итерации мы вариировали минимальный порог частоты начиная с 30 строк и заканчивая 100 строками.

Замеры времени происходили с учетом времени обработки таблицы. Чтобы минимизировать влияние операционной системы на производительность кеши ОС сбрасывались между каждым запуском с помощью записи "3" в /proc/sys/vm/drop_caches, на время экспериментов выключался файл подкачки с помощью команды swapoff -a, частота процессора фиксировалась и выставлялась в максимально возможное значение с помощью cpupower, все тяжеловесные процессы были отключены. Обе программы компилировались с флагом оптимизаций -O3.

[3] https://www.kaggle.com/datasets/likithagedipudi/campus-placement-prediction

6.2. Результаты экспериментов

Результаты экспериментов представлены на рисунках 2 и 3.

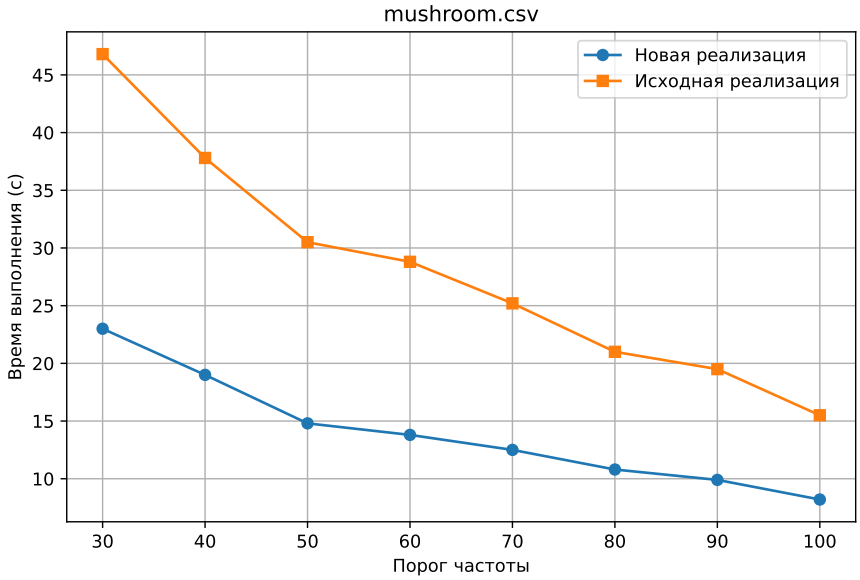

Рис. 2: Результаты тестирования на наборе данных mushroom.csv.

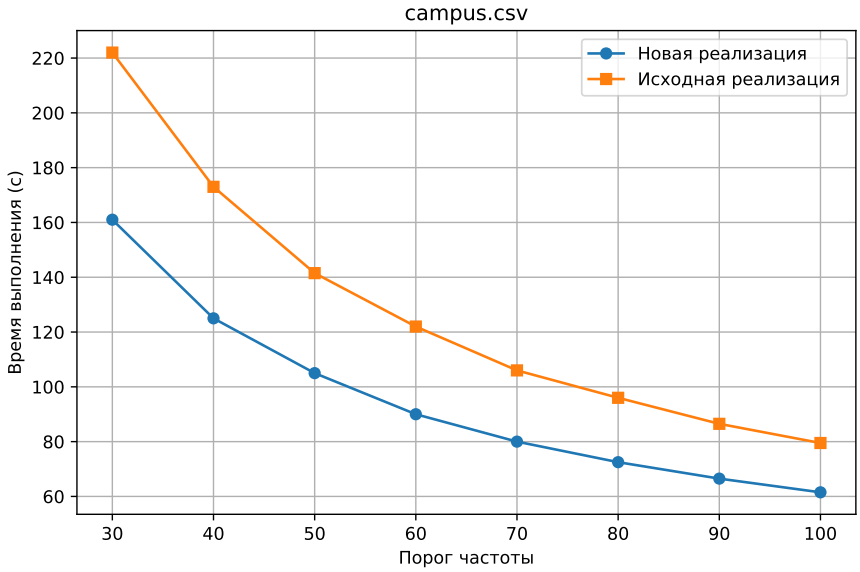

Рис. 3: Результаты тестирования на наборе данных campus.csv.

Из графиков видно, что несмотря на изменение способа представления множества атрибутов, новая реализация работает в 1.35–2.0 раз быстрее исходной. При этом обе версии заканчивают поиск намного раньше с ростом порога частоты, так как увеличение

порога частоты увеличивает число избыточных множеств, которые отсекаются на каждой итерации алгоритма.

Также было замечено, что на тестируемых наборах данных обе реализации большую часть времени тратят на вычисление квази-замыкания. Предположительно, это происходит из-за логики обновления квази-замыкания, которое в обоих реализациях для каждого кандидата текущего уровня выполняет тройной цикл по подмножествам кандидата и их кластерам. В связи с этим дальнейшие оптимизации алгоритма могут быть направлены на пересмотр указанного цикла.

7. Заключение

По итогам работы были достигнуты следующие результаты:

1. Проведен обзор предметной области для алгоритма CFUN;
2. Алгоритм CFUN интегрирован в платформу Desbordante на языке C++, были исправлены ошибки исходной реализации и расширены возможности алгоритма по обработке таблиц с большим количеством столбцов;
3. Проведено тестирование и сравнение полученной реализации и реализации из статьи [4].

Ссылка на pull request: https://github.com/Desbordante/desbordante-core/pull/692.

Список литературы

[1] Conditional Functional Dependencies for Data Cleaning / Philip Bohannon, Wenfei Fan, Floris Geerts et al. // 2007 IEEE 23rd International Conference on Data Engineering. — 2007. — P. 746–755.

[2] Cosmadakis Stavros S, Kanellakis Paris C, Spyratos Nicolas. Partition semantics for relations // J. Comput. Syst. Sci. — 1986. — Oct. — Vol. 33, no. 2. — P. 203–233. — URL: https://doi.org/10.1016/0022-0000(86)90019-X.

[3] Desbordante: from benchmarking suite to high-performance science-intensive data profiler / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: https://doi.org/10.1145/3703323.3703725.

[4] Diallo Thierno, Novelli Nöél, Petit Jean-Marc. Discovering (frequent) constant conditional functional dependencies // International Journal of Data Mining, Modelling and Management. — 2012. — Vol. 4, no. 3. — P. 205–223. — PMID: 48104. URL: https://www.inderscienceonline.com/doi/abs/10.1504/IJDMMM.2012.048104.

[5] Novelli Noël, Cicchetti Rosine. FUN: An Efficient Algorithm for Mining Functional and Embedded Dependencies // Database Theory — ICDT 2001 / Ed. by Jan Van den Bussche, Victor Vianu. — Berlin, Heidelberg : Springer Berlin Heidelberg, 2001. — P. 189–203.